

Support Vector Machines for Classification

Scott Nguyen

Dept. of Electrical and Computer Engineering

University of New Mexico

Albuquerque, NM, USA

scott.nguyen@example.edu

Abstract—The theory and use of Support Vector Machines (SVMs) for binary classification are presented in this paper. We examine the dual optimization problem and kernel-based extensions in the mathematical formulation of hard and soft margin SVMs, and we show how to use them practically with Python-generated synthetic data. The empirical findings demonstrate how data structure and parameter selection affect the classifier’s performance and generalization ability.

Index Terms—support vector machines, classification, machine learning

I. INTRODUCTION

Classification is a big part of many modern technologies. It can help with everything from sorting pictures and finding broken systems to diagnosing diseases. You need to use labeled examples to train a model that can correctly put new, unseen data into a category. You can use a number of different methods for this, such as logistic regression, neural networks, and decision trees. Each method can work, but they are not all the same when it comes to how well they generalize, how easy they are to understand, and how much computing power they need.

Cortes and Vapnik developed Support Vector Machines (SVMs), which strike a unique balance between these properties. SVMs look for a boundary between classes that maximizes the margin, which is the distance between the closest data points of each class and the separating hyperplane. Because SVMs prefer large margins, they are less likely to be affected by noise. Their training process avoids local minima because the underlying optimization problem is convex, and they are also less likely to overfit than decision trees, especially when working with datasets that have a lot of features compared to the number of samples.

Researchers are still looking into ways to improve SVMs beyond their basic form. For example, there are fuzzy and distributed SVMs that work with noisy or large datasets [2], and there are specialized kernel functions that work with discrete inputs [3]. Other work in bioinformatics has even focused on making SVMs easier to use so that researchers can use them without having to learn a lot of programming [4]. Even with these changes, linear SVMs are still very popular, especially when the data is almost linearly separable or when interpretability is important. They are also efficient during prediction because they only need a small number of influential samples, called support vectors.

In this paper, we implement and evaluate a linear SVM using a synthetic dataset constructed for Module 3 of this course. After reviewing the fundamental theory and the dual optimization framework introduced in the lectures, we describe the setup and results of our experiments. We then analyze how the choice of margin regularization affects performance and discuss the role of support vectors in shaping the decision boundary. We conclude by suggesting directions for improving the model and exploring more advanced techniques, such as kernel methods and hyperparameter optimization.

II. THEORY

Support Vector Machines (SVMs) are supervised classification models that aim to identify the optimal separating hyperplane between two classes of labeled data. Given a labeled dataset $\{(\mathbf{x}_n, y_n)\}_{n=1}^N$ with feature vectors $\mathbf{x}_n \in \mathbb{R}^d$ and binary labels $y_n \in \{-1, +1\}$, the Support Vector Classifier (SVC) assumes a linear separating function of the form

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b, \quad (1)$$

where $\mathbf{w}$ represents the normal vector to the hyperplane and b is the bias term.

A. Maximum Margin Principle

The key idea behind SVMs is the *maximum margin principle*: the classifier should not merely separate the two classes but do so with the largest possible distance (margin) between the decision boundary and the nearest data points. For perfectly separable data, the functional margin of each point is defined as $y_n(\mathbf{w}^T \mathbf{x}_n + b)$, and correct classification requires

$$y_n(\mathbf{w}^T \mathbf{x}_n + b) \geq 1. \quad (2)$$

The geometric margin follows as $2/\|\mathbf{w}\|$, making the maximization of margin equivalent to minimizing $\|\mathbf{w}\|$. This leads to the *primal optimization problem*:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{subject to} \quad y_n(\mathbf{w}^T \mathbf{x}_n + b) \geq 1. \quad (3)$$

B. Soft Margin Classification

Real-world data are often not linearly separable. To allow some classification errors while maintaining a large margin, *slack variables* $\xi_n \geq 0$ are introduced to relax the constraint:

$$y_n(\mathbf{w}^T \mathbf{x}_n + b) \geq 1 - \xi_n. \quad (4)$$

The resulting optimization problem is:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{n=1}^N \xi_n, \quad (5)$$

where $C > 0$ controls the penalty for margin violations. A large C forces the classifier to prioritize correct classification (possibly at the cost of generalization), while a small C prioritizes margin width over perfect separation.

C. Lagrangian and KKT Conditions

The optimization problem in (5) is solved by introducing non-negative Lagrange multipliers α_n and forming the Lagrangian:

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{n=1}^N \alpha_n (y_n (\mathbf{w}^T \mathbf{x}_n + b) - 1 + \xi_n). \quad (6)$$

Setting the derivative of $\mathcal{L}$ with respect to $\mathbf{w}$ to zero yields the optimal weight vector:

$$\mathbf{w} = \sum_{n=1}^N \alpha_n y_n \mathbf{x}_n. \quad (7)$$

This reveals that $\mathbf{w}$ is a linear combination of the training samples weighted by their labels and Lagrange multipliers.

Only samples with $\alpha_n > 0$ contribute to $\mathbf{w}$; these are the *support vectors*. Geometrically, they lie closest to the decision boundary and fully define the classifier. Removing any non-support vector leaves the decision boundary unchanged.

D. Dual Problem

By eliminating the primal variables, the *dual problem* becomes:

$$\max_{\boldsymbol{\alpha}} \sum_{n=1}^N \alpha_n - \frac{1}{2} \sum_{n,m=1}^N \alpha_n \alpha_m y_n y_m \mathbf{x}_n^T \mathbf{x}_m, \quad (8)$$

subject to:

$$0 \leq \alpha_n \leq C, \quad \sum_{n=1}^N \alpha_n y_n = 0. \quad (9)$$

Solving the dual yields the optimal α_n values, from which the classifier parameters $\mathbf{w}$ and b can be computed.

E. Summary

The SVC optimizes a convex objective that balances margin maximization and classification error. Its formulation ensures global optimality during training, and its reliance on support vectors provides a sparse and efficient representation for inference. This makes SVMs powerful and interpretable tools for classification, particularly in high-dimensional settings where simpler linear models may struggle.

III. EXPERIMENTS

To validate the theoretical foundations of the Support Vector Classifier (SVC) presented in Section II, we conducted a series of experiments as part of Module 3 in this course. Using the provided Python code, we trained and evaluated a linear SVM on synthetically generated data. The experiment was designed to highlight the effect of the margin regularization parameter C on classifier performance, and to observe the role of support vectors in determining the decision boundary.

A. Dataset

The dataset used in these experiments was generated using the provided `data()` function. Each sample $\mathbf{x}$ is a point in $\mathbb{R}^{10}$, and belongs to one of two classes ($y \in \{-1, +1\}$). The function generates structured clusters by perturbing fixed prototype vectors with Gaussian noise scaled by a parameter σ .

A total of $N = 100$ samples were used for training, and 1000 samples were independently generated for testing. The class distribution for both sets was balanced to avoid introducing bias during learning. Due to the additive noise and dimensionality of the feature space, the generated data is not linearly separable, making it suitable for testing soft-margin SVM behavior.

B. Classifier Setup

We employed the SVC class from the `scikit-learn` library with a linear kernel:

```
clf = svm.SVC(kernel="linear", C=0.01)
```

The hyperparameter $C = 0.01$ was chosen to enforce a wide margin by allowing more classification errors during training. According to the optimization framework described in Section II, lower values of C reduce the penalty for misclassification, leading the SVM to favor simpler decision boundaries that may generalize better.

To train the classifier, we called `clf.fit()` on the training data ($X_{\text{tr}}, y_{\text{tr}}$), and then used `clf.predict()` to obtain predictions on both the training and testing sets. Predictions were compared against true labels to compute empirical error estimates.

C. Reproducibility and Parameter Variation

To ensure reproducibility, all random number generation was controlled with a fixed random seed. Additionally, to observe the impact of the margin regularization parameter, we repeated the experiments for different values of C , specifically $C \in \{0.01, 0.1, 1.0\}$, while keeping all other settings fixed.

This allows us to analyze how the flexibility of the decision boundary, governed by C , affects performance. The classifier's dual coefficients and number of support vectors were also recorded to connect the empirical findings to the theory presented earlier.

D. Evaluation Metrics

We evaluated classification performance using the empirical error rate, measured as the proportion of misclassified samples:

$$E = \frac{1}{N} \sum_{n=1}^N \mathbb{I}\{y_n \neq \hat{y}_n\},$$

where $\mathbb{I}$ is the 0-1 indicator function, N is the number of samples in the evaluation set, and $\hat{y}_n$ is the predicted label for sample n .

In addition to error rates, we analyzed the dual coefficient values α_n computed during training to identify the support vectors. These values are particularly informative, as they determine which samples lie closest to the decision boundary and influence the shape of the classifier. As shown in Section IV, this analysis reveals the effect of C on the classifier complexity and generalization behavior.

IV. RESULTS AND DISCUSSION

The Support Vector Classifier was trained and evaluated on the synthetic dataset generated using $\sigma = 1$, as described in Section III. We conducted multiple runs varying the regularization parameter C to assess its impact on classifier flexibility, error rate, and support vector behavior.

A. Classification Error

Table I shows the empirical classification error rates for different values of C . The training set contained 100 samples, and the test set contained 1000 samples generated independently.

TABLE I
CLASSIFICATION ERROR FOR DIFFERENT VALUES OF C

C Value	Train Error	Test Error
0.01	0.48	0.50
0.1	0.41	0.46
1.0	0.31	0.42

As expected, increasing the value of C resulted in a decrease in both training and test error. This is consistent with the theoretical role of C as a margin regularization parameter. For small values of C , margin violations are less penalized, leading the classifier to prioritize a wide margin at the expense of accuracy. Conversely, larger values of C enforce stricter classification constraints, increasing the model's capacity to fit the training data but potentially risking overfitting.

B. Support Vectors and Dual Coefficients

In all experiments, the dual coefficients α_n appeared in small magnitudes on the order of 10^{-2} for $C = 0.01$, and increased slightly with larger C values. For $C = 1.0$, several coefficients reached magnitudes near 0.1. According to the theory in Section II, only samples with $\alpha_n > 0$ contribute to the classifier's decision boundary as support vectors.

Fig. ?? (omitted) shows a visualization of the α_n values for different C values. We observed that as C increased, a larger number of training samples contributed nonzero dual

coefficient values, indicating a more complex decision boundary. This is consistent with the intuition that larger C values attempt to reduce misclassification by pulling additional points onto or across the margin.

C. Discussion

The results validate key theoretical properties of SVMs: the balance between margin size and classification accuracy controlled by C , and the sparse representation of the classifier via support vectors. In this synthetic, non-linearly separable dataset, small values of C produced nearly random performance due to insufficient capacity, while larger values ($C = 1.0$) improved results without significant overfitting.

Compared to other supervised classifiers, such as logistic regression or neural networks, linear SVMs offer clear geometric interpretations and global optimality guarantees. However, their reliance on linear separability limits their performance in high-dimensional spaces unless kernel extensions are applied. In practice, further improvements may be achieved by:

- Introducing a nonlinear kernel (e.g., RBF, polynomial) to handle complex class boundaries;
- Conducting hyperparameter search (e.g., grid search over C and γ);
- Increasing the number of training samples or reducing noise in feature generation.

Overall, this experiment demonstrates the practical implications of theory and highlights the interplay between model capacity, margin regularization, and empirical performance.

V. RELATED WORK

The broader landscape of Support Vector Machine research spans multiple domains, building on the core theory presented in this work. Jakkula [1] provides one of the most accessible introductions to SVMs, emphasizing the mathematical foundation and optimization procedure involved in constructing separating hyperplanes. This work has served as a foundational reference in machine learning education.

Building on classical SVM theory, Wang *et al.* [2] present a comprehensive survey of SVM variants and their applications in multimedia classification. They highlight extensions such as fuzzy SVMs for handling uncertain labels and distributed SVMs for large-scale data, demonstrating the versatility and adaptability of the method in real-world environments.

Recent research continues to extend the functionality of SVMs. For example, Amaya-Tejera *et al.* [3] propose a novel distance-based kernel specifically designed for binary and categorical datasets, showing competitive or superior performance on benchmark problems. Other applied work, such as that of Pirooznia and Deng [4], demonstrates how SVMs can be integrated into graphical user interfaces to support non-expert users in bioinformatics contexts. Winters-Hilt *et al.* [5] further adapt SVMs for hybrid tasks involving both classification and clustering, illustrating the model's flexibility for signal processing and structural biology.

Together, these works underline the continued relevance of SVMs in both academic research and practical deployment,

affirming their importance as a core method for modern machine learning tasks.

VI. CONCLUSION

In this paper, we presented a study of Support Vector Machines (SVMs) for binary classification using the Support Vector Classifier (SVC) framework introduced in Module 3. We reviewed the theoretical foundations of SVMs, including the formulation of the hard and soft margin classifiers, the introduction of slack variables, and the associated dual optimization problem. The structure of the solution revealed the role of support vectors as the key determinants of the optimal separating hyperplane.

Through experiments conducted on a synthetic dataset, we demonstrated how the margin regularization parameter C influences classifier performance and model complexity. In particular, small values of C resulted in poorer performance due to insufficient classifier flexibility, while larger values allowed the model to fit the data more closely but still generalized reasonably well. The behavior of the dual coefficients further illustrated how support vectors govern the decision boundary as predicted by theory.

The linear SVM model used in this study is effective when the class boundaries are approximately linear and when model interpretability is a priority. However, its performance is limited on non-linearly separable data, as revealed by the near-random performance when C was small. These limitations suggest several directions for future work, including the incorporation of nonlinear kernel functions, tuning of hyperparameters via cross-validation, and expanding the dataset to improve robustness.

Overall, this study reinforces the value of integrating theoretical understanding with empirical validation. Support Vector Machines remain a powerful and interpretable tool for classification problems, and this work highlights both their strengths and the conditions under which they may require extension or modification.

REFERENCES

- [1] V. Jakkula, "Tutorial on support vector machine (SVM)," School of EECS, Washington State University, Tech. Rep., 2006.
- [2] H. Wang *et al.*, "A research survey on support vector machine," *International Journal of Multimedia and Ubiquitous Engineering*, vol. 8, no. 2, pp. 1–16, 2017.
- [3] N. Amaya-Tejera, P. Melin, O. Castillo, and A. Mendoza, "A distance-based kernel for classification via support vector machines," *Neurocomputing*, vol. 556, p. 126630, 2024.
- [4] M. Pirooznia and Y. Deng, "SVM Classifier—a comprehensive Java interface for support vector machine classification of microarray data," *BMC Bioinformatics*, vol. 7, no. 4, pp. 1–8, 2006.
- [5] S. Winters-Hilt *et al.*, "Support vector machine implementations for classification and clustering," *BMC Bioinformatics*, vol. 7, suppl. 2, p. S4, 2006.